SNT

Interdisciplinary Centre for
Security, Reliability and Trust

Microkernel-Based Systems

Marcus Völp

uni.lu
UNIVERSITY OF
LUXEMBOURG

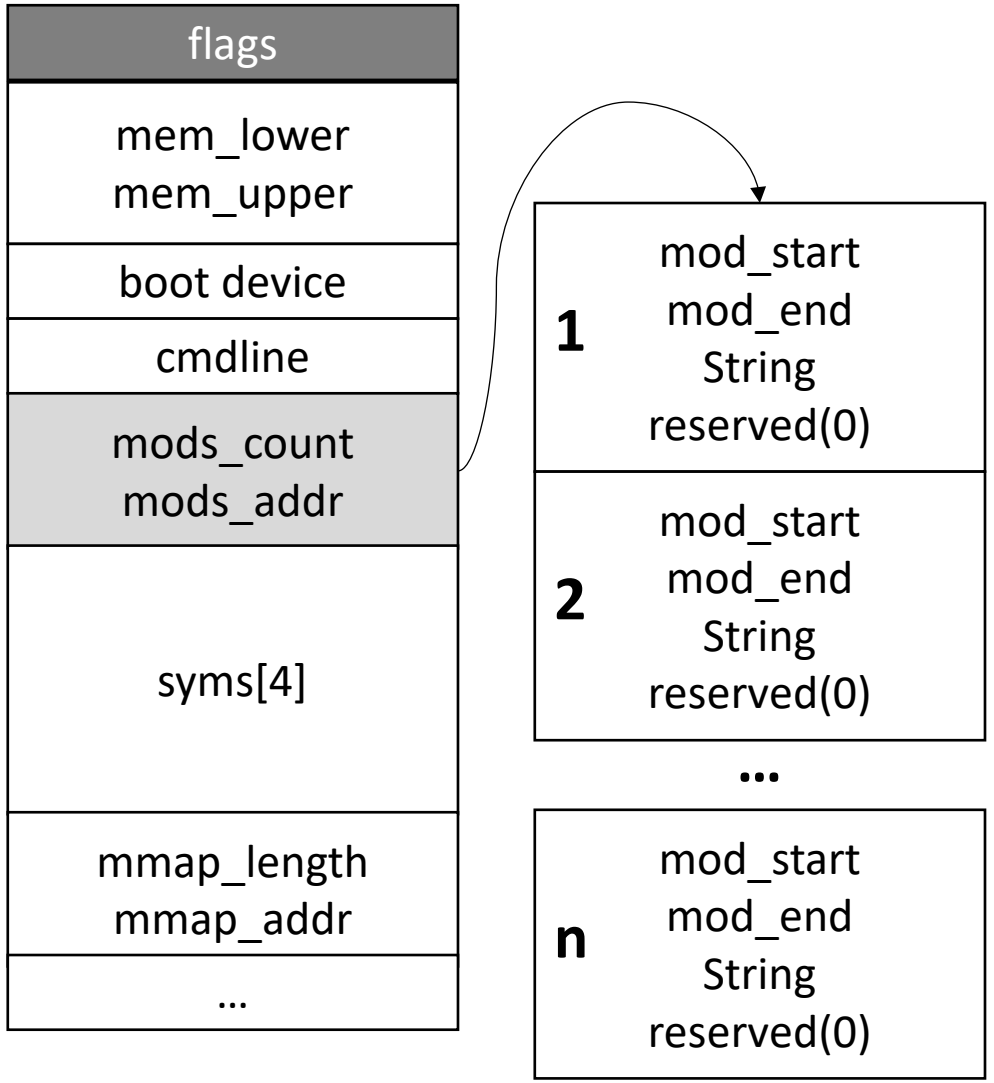
Practical Assignments II + III – Marked!!

based on Nils' Exercise

- we will skip elf loading / multiboot
- first two systemcalls
  - create thread (aka create_ec)
  - yield (cooperatively switch to the other thread)
- mkc_ex3
  - root invoke: loads elf file
  - mkc/user/…: user coder (no more need for address magic)
  - ec.cc => Ec::syscall_handler: first syscall already present: dump
- deadlines:
  - ***To be discussed in next lecture***

What we skipped:

- multiboot information (multiboot.h)
- flags are mandatory, rest optional
- flag 3 is set to indicate mod_count + mod_addr is valid
- mod_addr: physical address to array of model structs with length mods_count
- if you use grub, pass binaries as modules: see grub docu + boot/menu.lst



CRITIX

What we skipped:

```
class Multiboot
{
    public:
        enum
        {
            MAGIC           = 0x2badb002,
            MEMORY          = 1ul << 0,
            BOOT_DEVICE     = 1ul << 1,
            CMDLINE         = 1ul << 2,
            MODULES         = 1ul << 3,
            SYMBOLS         = 1ul << 4 | 1ul << 5,
            MEMORY_MAP      = 1ul << 6,
            DRIVES          = 1ul << 7,
            CONFIG_TABLE    = 1ul << 8,
            LOADER_NAME     = 1ul << 9,
            APM_TABLE       = 1ul << 10,
            VBE_INFO        = 1ul << 11
        };
```

```
        uint32  flags;              // 0
        uint32  mem_lower;          // 4
        uint32  mem_upper;          // 8
        uint32  boot_device;        // 12
        uint32  cmdline;            // 16
        uint32  mods_count;         // 20
        uint32  mods_addr;          // 24
        uint32  syms[4];            // 28,32,36,40
        uint32  mmap_len;           // 44
        uint32  mmap_addr;          // 48
        uint32  drives_length;      // 52
        uint32  drives_addr;        // 56
        uint32  config_table;       // 60
        uint32  loader_name;        // 64
};
```

```
/*
 * Multiboot Module
 */
class Multiboot_module
{
    public:
        uint32  mod_start;
        uint32  mod_end;
        uint32  cmdline;
        uint32  reserved;
};
```

CritiX

What we skipped:

```
    // find multi boot info
    Multiboot * mbi = static_cast<Multiboot *>(Ptab::remap (current->regs.eax));

    if (!(mbi->flags & 8) || (mbi->mods_count != 1))
        panic ("exactly ONE multi boot module is required.\n");

    // load module desciptor
    Multiboot_module mod = *static_cast<Multiboot_module *>(Ptab::remap (mbi->mods_addr));

    printf ("load module from %x - %x (%u bytes) : ", mod.mod_start, mod.mod_end, mod.mod_end - mod.m
od_start);
    char * cmd = static_cast<char *>(Ptab::remap (mod.cmdline));
    printf ("%s\n",cmd);

    // remap elf header
    Eh * e = static_cast<Eh *>(Ptab::remap (mod.mod_start));
    if (e->ei_magic != 0x464c457f || e->ei_data != 1 || e->type != 2)
        panic ("No ELF\n");

    unsigned count = e->ph_count;
    current->regs.eip = e->entry;
```

What we skipped:

```
// remap program headers
Ph * p = static_cast<Ph *>(Ptab::remap (mod.mod_start + e->ph_offset));

for (; count--; p++) {

    if (p->type == Ph::PT_LOAD) {

        unsigned attr = p->flags & Ph::PF_W ? 7 : 5;

        if (p->f_size != p->m_size || p->v_addr % PAGE_SIZE != p->f_offs % PAGE_SIZE)
            panic ("Bad ELF\n");

        mword phys = align_dn (p->f_offs + mod.mod_start, PAGE_SIZE);
        mword virt = align_dn (p->v_addr, PAGE_SIZE);
        mword size = align_up (p->f_size, PAGE_SIZE);

        while (size) {
            Ptab::insert_mapping (virt, phys, attr);
            virt += PAGE_SIZE;
            phys += PAGE_SIZE;
            size -= PAGE_SIZE;
        }
    }
}
```

CRITIX

Exercise II: (Deadline tba)

- **Task 1: first simple scheduler (40%)**
  - Implement a ready list to hold all ready-to-execute ECs
  - The list should at least allow dequeueing the first ready to execute thread and enqueing at the tail (uniprocessor solutions are fine)
  - Take into consideration where you allocate the memory blocks for the list
  - Also, make sure to establish invariants for both newly created ECs and the one that is created by the kernel as the first thread (the one executing root_invoke)
  - Implement a create EC systemcall to create a new thread in the same address space. The user should specify the user-level instruction and stack pointers that the thread will use.
  - Implement a yield system call to relinguish the CPU while remaining ready to execute. When yielding, the thread should enqueue itself to the tail of the ready list, giving preference to other threads in that list.
  - Test your code and create sufficient debug output.

CRITIX

Exercise II: (Deadline tba)

- **Task 2: extend the scheduler to a fixed priority one (30%)**
  - Extend the ready list to support a limited number of priorities
  - Extend create_EC to take a priority value
  - ECs with the same priority should be scheduled round robin (i.e., dequeue front, enqueue tail), otherwise, the highest priority EC should run
  - Create 4 threads at two different priorities (2 each)
  - Test your code and create sufficient debug output.

CRITIX

Exercise II: (Deadline tba)

- **Task 3: blocking (30%)**
  - Implement a second list to hold blocking ECs
  - Create a system call to block an EC (blocked Ecs are no longer ready to execute)
  - Create a system call to unblock all ECs in the blocking list
  - Test your code and create sufficient debug output.

  - Advise: keep the initial EC at the lowest priority and let this thread invoke the system call to unblock the others.

CRITIX

Exercise III): (Deadline tba)

- **Task 4: Capabilities (30%)**
  - Implement a simple capability system for Ecs
  - Extend create_EC to take a capability slot
  - Maintain a data structure in the kernel so that each protection domain can find kernel objects that are created, giving the slot number they specify

CRITIX

Exercise III): (Deadline tba)

- **Task 5: IPC (50%)**
  - Assign all ECs the same priority or start from the single priority code
  - Conceive a simple IPC system call to send and receive a single register value
  - IPC should be synchronous (i.e., ECs that find a partner not to be ready to receive / ready to send should be blocked on that EC).
  - Test your code with multiple ECs sending to / receiving from a single EC; create sufficient debug output.
  - Take care of the scheduling that happens when an EC becomes blocked.

CRITIX

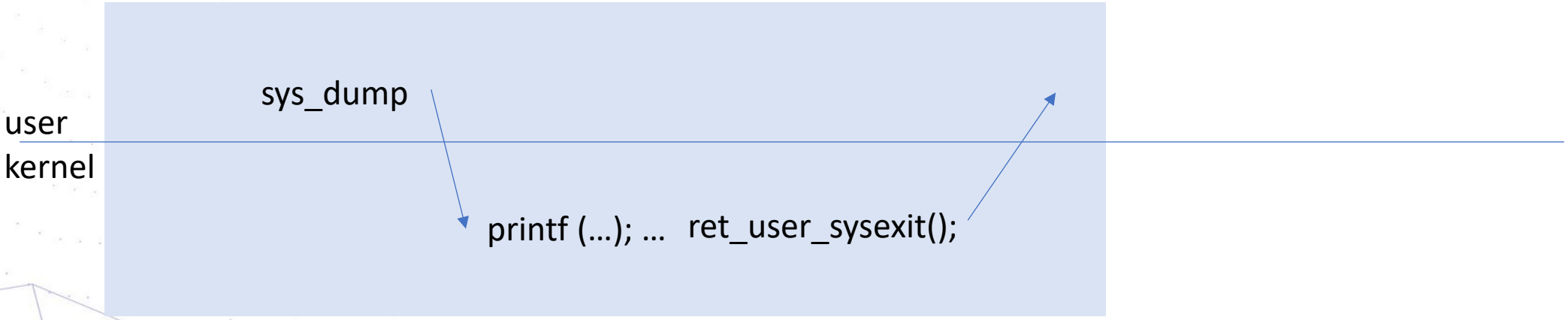
Exercise III): (Deadline tba)

- **Task 6: IPC with priorities (20%)**
  - Extend your code to work with priorities.
  - Conceive and describe a solution to deal with priority inversion

CritiX

First two real Systemcalls

- „normal“ systemcall



CritiX

First two real Systemcalls

- yield systemcall